

due to the loss of orthogonality of basis vectors. Thus we set the maximum iteration steps to $10n$. We have observed that for the test problems in our experiments, $10n$ is enough to ensure convergence. We perform full reorthogonalization on V'_k , U_k and $\hat{U}_k$ to ensure that they are numerically orthonormal to the working precision ϵ . By default, the convergence of line 2 in Algorithm 6.1 is judged by (3.18) with the stopping tolerance $tol = 10^{-8}$. The starting vector u_1 is randomly generated by the standard normal distribution function `randn` in MATLAB and then normalized.

Parameters	Default values	Description
<i>target</i>	5	$ target = l$ is the number of desired GSVD components, where $target > 0$ or $target < 0$ means that the largest or smallest GSVD components are required.
$k_{\max}$		Maximum subspace dimension
<i>adjust</i>	3	Integer added to l to speed up convergence
<i>tol</i>	10^{-8}	Stopping tolerance
<i>maxit</i>	1000	Maximum number of restarts
<i>lsqrtol</i>	10ϵ	Stopping tolerance of <code>lsqr</code>
<i>lsqrmaxit</i>	$10n$	Maximum number of iterations of <code>lsqr</code>
u_1	generated by <code>randn</code> and then normalized	The unit length starting vector

TABLE 1
Parameters of IRJBD

7. Numerical experiments. We now present a number of numerical experiments to demonstrate the performance of Algorithm 6.1, written in MATLAB language, and compare it with the public C language code of TRJBD in [1]. The experiments will illustrate that Algorithm 6.1 is at least competitive with and can outperform TRJBD considerably in terms of restarts. Since section 4.4 has shown that each restart of IRJBD and TRJBD basically costs the same for the same l and $k_{\max}$, fewer restarts mean higher efficiency. The experiments were performed on an AMD Ryzen 7 5800X CPU with 78 GB RAM and 8 cores using the MATLAB R2024a with the machine precision $\epsilon = 2.22 \times 10^{-16}$ under the Ubuntu 20.04.3 LTS 64-bit system.

We use some matrices from the SuiteSparse Matrix Collection [6] as A , and L is taken as (cf. [11]):

$$L = \begin{pmatrix} 3 & 1 & & & \\ 1 & \ddots & \ddots & & \\ & \ddots & \ddots & 1 & \\ & & & 1 & 3 \end{pmatrix},$$

which is well conditioned. Table 2 lists the test matrices A and some of their properties, where nnz is the total number of nonzero entries of $\{A, L\}$ and $\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$, available from the SuiteSparse Matrix Collection, is the condition number of A and indicates the level of ill-conditioning of A and reflects magnitudes of generalized singular values of $\{A, L\}$. In order to ensure that all the A are flat or square, i.e., $m \leq n$, we transpose some matrices, denoted by superscript T. Note that the smallest singular value of `kneser_8_3_1`^T is tiny, which causes that the matrix is numerically row